OOM-Protect — Project Constitution

- Article I — Anti-Bluff Testing (Non-Negotiable)
  - Mandatory anti-bluff rituals
  - Examples of bluffs that are forbidden
- Article II — Idempotency and Reversibility
- Article III — Documentation Parity
- Article IV — Observable Behavior Over Code Beauty
- Article V — Multi-Upstream Discipline
- Article VI — Submodule Constitutions
- Amendment process

OOM-Protect — Project Constitution

This document is the **highest-authority charter** for the OOM-Protect project and every submodule, tool, script, and document it contains. Every contributor — human or AI — MUST read and obey it. When this Constitution conflicts with any other guidance (CLAUDE.md, AGENTS.md, READMEs, tickets, ad-hoc requests), **the Constitution wins** unless the conflict is resolved by an explicit, documented amendment to this file.

The CLAUDE.md and AGENTS.md files in this repository, and in every current and future submodule, MUST embed Article I (Anti-Bluff Testing) verbatim or by reference.

Article I — Anti-Bluff Testing (Non-Negotiable)

> **A passing test or Challenge MUST prove that the product really works. A test that passes while the product is broken is a defect more serious than the original bug.**

We were burned. Tests “passed”, Challenges “ran green”, and the product did not work. That is forbidden going forward.

Concretely, every test and every Challenge MUST satisfy ALL of the following:

1. **Exercise the real artifact, not a stand-in.** Unit tests may use stubs for *external* boundaries (the kernel, atop, the network). They MUST NOT stub the code under test. Integration tests and Challenges MUST run the actual built binary against real inputs and observe real outputs.
2. **Assert observable outcomes, not internal state.** Verify files exist on disk with the required content; verify exit codes; verify subprocess stdout/stderr; verify systemd units transition to expected states. “I called the function” is not an assertion.
3. **Fail loudly when the feature is missing.** A test that passes vacuously when its target code is deleted is a bluff. Every test MUST be one whose green status genuinely depends on the production code being correct. Use mutation-style spot checks: temporarily break the code and confirm the test goes red.
4. **No `t.Skip`, no `pending`, no commented-out assertions, no `|| true`, no `--no-fail`, no swallowed exit codes.** A test that cannot run is not a test. If a precondition is missing, the test MUST fail with a clear diagnostic, not silently pass.
5. **Challenges MUST simulate the real failure mode.** The point of a Challenge is to reproduce the production hazard the feature defends against. A memory-protection Challenge MUST allocate enough memory to trip the threshold; a CPU Challenge MUST burn enough CPU; a startup Challenge MUST actually start the daemon. Synthetic shortcuts that bypass the hazard are forbidden.
6. **Every public feature has at least one Challenge.** No feature ships without an end-to-end Challenge that an end user could reproduce by reading the script. If a feature cannot be exercised by a Challenge, it is not a feature — it is a fiction.

7. **The full test+Challenge suite is part of the definition of done.** A change is not complete until `make test` AND `make challenges` (or the equivalent) pass on a clean checkout, on the target host, with the same toolchain the user will use.

Mandatory anti-bluff rituals

- **Before declaring a task done**, the contributor MUST run the relevant tests AND the relevant Challenges and paste the output (or a verifiable summary) in the change log.
- **Every PR / merge** MUST list the Challenges exercised and their outcomes.
- **Each Challenge script** MUST print, on success, what it verified — not just `OK`. Example: `OK: report file /var/log/oom-watch/reports/2026-04-30T14:23-memory.md exists, contains 'CRITICAL', and lists at least 5 PIDs`. A bare `OK` is forbidden.
- **Bluff audits**: periodically (at least once per release) a contributor MUST temporarily break a representative production line and confirm the corresponding test goes red. If it stays green, that test is a bluff and MUST be rewritten before the release ships.

Examples of bluffs that are forbidden

- A unit test that calls `parseAtop("")` and asserts `err == nil` without inspecting the parsed value.
- A Challenge that runs the binary with `--help` and exits 0, claiming the feature works.
- A test that asserts `len(reports) >= 0` (always true).
- A test that mocks the function it is supposed to test.
- A `make test` target that pipes through `|| true`, swallowing real failures.
- A CI green badge produced by a job that didn't actually execute the suite.

Article II — Idempotency and Reversibility

Every script that mutates the system MUST be:

1. **Idempotent.** Re-running it MUST be safe and produce the same end state.
2. **Reversible.** It MUST back up anything it overwrites, print the rollback command, and offer an `--uninstall` or `--rollback DIR` mode.
3. **Conservative on shared state.** Never restart `systemd-logind`, never edit `/etc/fstab`, swap, GRUB, or anything that could prevent the next boot.

These are non-negotiable for any tool in this repo, including new ones.

Article III — Documentation Parity

For every user-facing tool, the repository MUST contain:

1. A Markdown manual under `manuals/` or `docs/`.

2. The same content rendered to standalone HTML (CSS embedded).
3. The same content rendered to PDF.

`build-docs.sh` and `make docs` MUST produce all three from the Markdown source. Stale HTML/PDF is a bug.

Article IV — Observable Behavior Over Code Beauty

When in doubt, prefer code that emits clear, structured logs and writes legible artifacts on disk over code that is internally elegant but opaque. The user investigating a 3 a.m. incident will not read your structs; they will read your logs and your reports.

Article V — Multi-Upstream Discipline

The project is mirrored to four Git upstreams (GitHub, GitLab, GitFlic, GitVerse). The `origin` remote is configured to push to all four. Every commit on `main` MUST be pushed to all four; partial mirrors are forbidden because contributors and CI on different remotes will diverge.

Article VI — Submodule Constitutions

Any submodule added to this repository MUST contain its own `Constitution.md`, `CLAUDE.md`, and `AGENTS.md`. The submodule's Constitution MAY add rules but MUST NOT weaken Articles I–V of this Constitution. If a submodule lacks these files at the time of inclusion, the contributor adding the submodule MUST create them in the same commit.

Amendment process

This file is amended by an explicit commit whose message starts with `Constitution:` and which lists, in the body, the article changed and the rationale. No silent edits.

Adopted 2026-04-30, in response to repeated bluff-test incidents and to the OOM kill of the systemd user-manager on host `nezha` on 2026-04-28.